

Week-5 (Part-B)

1. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

OUTPUT:

```
adarohan18@Rohith:~$ nano Var_Exp.c
adarohan18@Rohith:~$ gcc Var_Exp.c -o Var_Exp
adarohan18@Rohith:~$ ./Var_Exp
Enter variable name: Rohan
Enter variable value: Hello
Enter text: Hello $Rohan
Expanded Value: Hello Hello
adarohan18@Rohith:~$ |
```

2. To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

Output:

```
adarohan18@Rohith:~$ nano Built_In_Commands.c
adarohan18@Rohith:~$ gcc Built_In_Commands.c -o Built_In_Commands
adarohan18@Rohith:~$ ./Built_In_Commands

Enter command: pwd
Current Directory: /home/adarohan18

Enter command: show
Built-in command executed successfully.

Enter command: help
Available commands:
pwd
show
help
exit

Enter command: exit
Program terminated.
```

Week-6 (Part-B)

1. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

Output:

```
adarohan18@Rohith:~$ nano BuiltIn_Commands.c
adarohan18@Rohith:~$ gcc BuiltIn_Commands.c -o BuiltIn_Commands
adarohan18@Rohith:~$ ./BuiltIn_Commands

Enter command: pwd
Current Directory: /home/adarohan18

Enter command: show
Built-in command executed successfully.

Enter command: hello
Invalid command: hello

Enter command: exit
Program terminated.
```

2. To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

Output:

```
adarohan18@Rohith:~$ nano Change_Directory.c
adarohan18@Rohith:~$ gcc Change_Directory.c -o Change_Directory
adarohan18@Rohith:~$ ./Change_Directory
Current Directory: /home/adarohan18
Enter directory to change: /home
Previous Directory: /home/adarohan18
New Directory: /home
```

WEEK-7 (Part-B)

1.To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

Output:

```
adarohan18@Rohith:~$ nano pwd.c
adarohan18@Rohith:~$ gcc pwd.c -o pwd
adarohan18@Rohith:~$ ./ped
-bash: ./ped: No such file or directory
adarohan18@Rohith:~$ ./pwd
Current Directory: /home/adarohan18
Press Enter to exit...

Cleaning up resources...
adarohan18@Rohith:~$ |
```

2.To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

Output:

```
adarohan18@Rohith:~$ nano ProcessSync.c
adarohan18@Rohith:~$ gcc ProcessSync.c -o ProcessSync
adarohan18@Rohith:~$ ./ProcessSync
Parent Process Started.
Waiting for child process...
Child Process Started.
Child PID: 724
Child Process Completed.
Child process exited normally.
Child exit status: 10
Parent Process Completed.
adarohan18@Rohith:~$ |
```

WEEK-8

1.To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

Output:

```
adarohan18@Rohith:~$ nano CmdHistory.c
adarohan18@Rohith:~$ gcc CmdHistory.c -o CmdHistory
adarohan18@Rohith:~$ ./CmdHistory
```

1. Add Command
2. Display History
3. Exit

Enter choice: 1

Enter command: ls

Command stored successfully.

1. Add Command
2. Display History
3. Exit

Enter choice: 1

Enter command: pwd

Command stored successfully.

1. Add Command
2. Display History
3. Exit

Enter choice: 1

Enter command: gcc CmdHistory.c -o CmdHistory

Command stored successfully.

1. Add Command
2. Display History
3. Exit

Enter choice: 2

Command History:

- 1 ls
- 2 pwd
- 3 gcc CmdHistory.c -o CmdHistory

1. Add Command
2. Display History
3. Exit

Enter choice: 3

2.To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

Output:

```
adarohan18@Rohith:~$ nano PipeLineStruc.c
adarohan18@Rohith:~$ gcc PipeLineStruc.c -o PipeLineStruc
adarohan18@Rohith:~$ ./PipeLineStruc
Enter pipeline commands using | : ls | grep .c | wc -l

Pipeline Structure:
Command: ls Command:  grep .c Command:  wc -l

Pipeline order validated.
adarohan18@Rohith:~$ |
```

3.To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

Output:

```
adarohan18@Rohith:~$ nano SinglePipe.c
adarohan18@Rohith:~$ gcc SinglePipe.c -o SinglePipe
adarohan18@Rohith:~$ ./SinglePipe
103
Data flow completed through pipe.
adarohan18@Rohith:~$ |
```

4.To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

Output:

```
adarohan18@Rohith:~$ nano MultiPipes.c
adarohan18@Rohith:~$ gcc MultiPipes.c -o MultiPipes
adarohan18@Rohith:~$ ./MultiPipes
60
Multiple-pipe pipeline completed.
adarohan18@Rohith:~$ |
```

WEEK-9

1. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

Output:

```
adarohan18@Rohith:~$ nano InpRedirect.c
adarohan18@Rohith:~$ gcc InpRedirect.c -o InpRedirect
adarohan18@Rohith:~$ ./InpRedirect
Reading from input.txt:
Hello, this is my input file.
I am learning Linux system calls.
This file will be read using the read() system call.
adarohan18@Rohith:~$ |
```

2. To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

Output:

```
adarohan18@Rohith:~$ nano OutputRedirect.c
adarohan18@Rohith:~$ gcc OutputRedirect.c -o OutputRedirect
adarohan18@Rohith:~$ ./OutputRedirect
Output stream restored.
```

3. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

Output:

```
adarohan18@Rohith:~$ nano AppendRedirect.c
adarohan18@Rohith:~$ gcc AppendRedirect.c -o AppendRedirect
adarohan18@Rohith:~$ ./AppendRedirect
Append operation completed.
adarohan18@Rohith:~$ |
```

4. To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

Output:

```
adarohan18@Rohith:~$ nano ErrorRedirect.c
adarohan18@Rohith:~$ gcc ErrorRedirect.c -o ErrorRedirect
adarohan18@Rohith:~$ ./ErrorRedirect
Error stream restored.
adarohan18@Rohith:~$ |
```

WEEK-10

1. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

Output:

```
adarohan18@Rohith:~$ nano CombinedRedirec.c
adarohan18@Rohith:~$ gcc CombinedRedirec.c -o CombinedRedirec
adarohan18@Rohith:~$ ./CombinedRedirec
adarohan18@Rohith:~$ cat combined.txt
Standard error message.
Standard output message.
adarohan18@Rohith:~$ |
```

2. To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

Output:

```
adarohan18@Rohith:~$ nano ComplexCmd.c
adarohan18@Rohith:~$ gcc ComplexCmd.c -o ComplexCmd
adarohan18@Rohith:~$ ./ComplexCmd
Complex command completed.
adarohan18@Rohith:~$ |
```

3. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

Output:

```
adarohan18@Rohith:~$ nano BackgroundJobs.c
adarohan18@Rohith:~$ gcc BackgroundJobs.c -o BackgroundJobs
adarohan18@Rohith:~$ ./BackgroundJobs
Prompt returned immediately.
Background job PID = 960
Background job started. PID = 960
Job is still running.
Background job completed.
Background job finished.
adarohan18@Rohith:~$ |
```

4. To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

Output:

```
adarohan18@Rohith:~$ nano JobTable.c
adarohan18@Rohith:~$ gcc JobTable.c -o JobTable
adarohan18@Rohith:~$ ./JobTable

Job Table
-----
ID      PID      State
1       1005     Running
2       1006     Running

Updated Job Table:

Job Table
-----
ID      PID      State
1       1005     Completed
2       1006     Running
adarohan18@Rohith:~$ |
```

WEEK-11

1. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

Output:

```
adarohan18@Rohith:~$ nano ListActiveJobs.c
adarohan18@Rohith:~$ gcc ListActiveJobs.c -o ListActiveJobs
adarohan18@Rohith:~$ ./ListActiveJobs
Active Jobs
-----
Job ID  PID      Status
[1]     1000     Running
[2]     1001     Running
[3]     1002     Running

Job listing completed successfully.
adarohan18@Rohith:~$ |
```

2. To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

Output:

```
adarohan18@Rohith:~$ nano ForegroundSwitching.c
adarohan18@Rohith:~$ gcc ForegroundSwitching.c -o ForegroundSwitching
adarohan18@Rohith:~$ ./ForegroundSwitching
Target job identified: PID = 1167
Waiting for foreground job...
Foreground job started. PID = 1167
Foreground job completed.
Job state updated: Completed
Terminal control returned to parent.
adarohan18@Rohith:~$ |
```

3. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

Output:

```
adarohan18@Rohith:~$ nano SiganlDelivery.c
adarohan18@Rohith:~$ gcc SiganlDelivery.c -o SiganlDelivery
adarohan18@Rohith:~$ ./SiganlDelivery
PID = 1267
Testing signal delivery and signal mask.
Blocking SIGUSR1...
SIGUSR1 is currently blocked.
Unblocking SIGUSR1...
Signal mask restored.
Process Group ID = 1267
Terminal signal concepts demonstrated.
adarohan18@Rohith:~$ |
```

4. To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

Output:

```
adarohan18@Rohith:~$ nano ResumeStoppedJobs.c
adarohan18@Rohith:~$ gcc ResumeStoppedJobs.c -o ResumeStoppedJobs
adarohan18@Rohith:~$ ./ResumeStoppedJobs
Job started. PID = 1316
Job status: Stopped
Sending SIGCONT...
Job resumed using SIGCONT.
Job completed.
Job status: Completed
adarohan18@Rohith:~$ |
```

WEEK-12

1. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

Output:

```
adarohan18@Rohith:~$ nano RegisSigHandlers.c
adarohan18@Rohith:~$ gcc RegisSigHandlers.c -o RegisSigHandlers
adarohan18@Rohith:~$ ./RegisSigHandlers
Signal handlers registered.
PID = 1370
Press Ctrl+C to test SIGINT.
^C
Signal 2 received.
Handler executed successfully.
```

2. To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

Output:

```
adarohan18@Rohith:~$ nano SIG_INT_HANDLRS.c
adarohan18@Rohith:~$ gcc SIG_INT_HANDLRS.c -o SIG_INT_HANDLRS.c
gcc: fatal error: input file 'SIG_INT_HANDLRS.c' is the same as output file
compilation terminated.
adarohan18@Rohith:~$ gcc SIG_INT_HANDLRS.c -o SIG_INT_HANDLRS
adarohan18@Rohith:~$ ./SIG_INT_HANDLRS
Press Ctrl+C to terminate the foreground job.
Foreground job running. PID = 1475
^C
SIGINT received. Forwarding to foreground job.
Foreground job terminated.
Job state updated.
adarohan18@Rohith:~$ |
```

3. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

OUTPUT:

```
adarohan18@Rohith:~$ nano SIGTSTP_Handl.c
adarohan18@Rohith:~$ gcc SIGTSTP_Handl.c -o SIGTSTP_Handl
adarohan18@Rohith:~$ ./SIGTSTP_Handl
Press Ctrl+Z to suspend the process.
Process running. PID = 1525
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
Process working...
```

```
Process working...
Process working...
Process working...
Process working...
^Z
SIGTSTP received.
Process working...

SIGTSTP received.
Job Table: Job is Stopped.
Resuming job with SIGCONT...
Job Table: Job is Running.
Job completed.
adarohan18@Rohith:~$ |
```

4. To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

Output:

```
adarohan18@Rohith:~$ nano ProcessGroups.c
adarohan18@Rohith:~$ gcc ProcessGroups.c -o ProcessGroups
adarohan18@Rohith:~$ ./ProcessGroups
Process 1: PID=598 PGID=598
Process 2: PID=599 PGID=598
Process group created.
Group ID = 598
Sending signal to complete process group...
Process group completed.
Terminal control can now be restored.
adarohan18@Rohith:~$ |
```

WEEK-13

1. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

Output:

```
adarohan18@Rohith:~$ nano ConnectModules.c
adarohan18@Rohith:~$ gcc ConnectModules.c -o ConnectModules
adarohan18@Rohith:~$ ./ConnectModules
Enter two numbers: 10 5
Module 1 - Addition = 15
Module 2 - Multiplication = 50
Interfaces connected successfully.
End-to-end execution completed.
adarohan18@Rohith:~$ |
```

2. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

Output:

```
adarohan18@Rohith:~$ nano RuntimeErrors.c
adarohan18@Rohith:~$ gcc RuntimeErrors.c -o RuntimeErrors
adarohan18@Rohith:~$ ./RuntimeErrors
1. Division
2. Invalid Syntax
Enter choice: 1
Enter two numbers: 18 45
Result = 0
Error scenario handled successfully.
adarohan18@Rohith:~$ |
```

3. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

Output:

```
adarohan18@Rohith:~$ nano ValgrindMemoCheck.c
adarohan18@Rohith:~$ gcc ValgrindMemoCheck.c -o ValgrindMemoCheck
adarohan18@Rohith:~$ ./ValgrindMemoCheck
Allocated values: 1 2 3 4 5
Memory released successfully.
Use Valgrind with:
valgrind --leak-check=full ./Week13_Q3_ValgrindMemoryCheck
adarohan18@Rohith:~$ |
```

4. To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

Output:

```
adarohan18@Rohith:~$ nano TestAutomation.c
adarohan18@Rohith:~$ gcc TestAutomation.c -o TestAutomation
adarohan18@Rohith:~$ ./TestAutomation
Test 1: PASS
Test 2: PASS
Test 3: PASS
Test 4: PASS

Test Report
Passed: 4/4
Failed: 0/4
```

WEEK-14

1. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

Output:

```
adarohan18@Rohith:~$ nano LargePipelines.c
adarohan18@Rohith:~$ gcc LargePipelines.c -o LargePipelines
adarohan18@Rohith:~$ ./LargePipelines
152
Pipeline completed.
Multiple jobs launched and monitored.
Execution completed successfully.
adarohan18@Rohith:~$ |
```

2. To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

Output:

```
adarohan18@Rohith:~$ nano ArchitectureDocument.c
adarohan18@Rohith:~$ gcc ArchitectureDocument.c -o ArchitectureDocument
adarohan18@Rohith:~$ ./ArchitectureDocument
OSSP Shell Architecture
=====

User Input
|
v
Command Parser
|
v
Built-in / External Command
|
v
Process Creation - fork()
|
v
Program Execution - exec()
|
v
Process Control - wait()

Important System Calls:
fork() - Create process
exec() - Execute program
wait() - Wait for process
pipe() - Create communication channel
dup2() - Redirect streams

Architecture and execution path documented.
adarohan18@Rohith:~$ |
```

3. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

Output:

```
adarohan18@Rohith:~$ nano SourceBuildRepository.c
adarohan18@Rohith:~$ gcc SourceBuildRepository.c -o SourceBuildRepository
adarohan18@Rohith:~$ ./SourceBuildRepository
Project Repository Structure
-----
src/          -> Source code
include/      -> Header files
build/        -> Build files
README.md     -> Documentation
Makefile      -> Build script

Build command:
gcc main.c -o main

Repository structure validated.
Source organization completed.
Deliverables are ready for release.
adarohan18@Rohith:~$ |
```

4. To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

Output:

```
adarohan18@Rohith:~$ nano FinalProjectDemo.c
adarohan18@Rohith:~$ gcc FinalProjectDemo.c -o FinalProjectDemo
adarohan18@Rohith:~$ ./FinalProjectDemo
Final Project Demonstration
=====

1. Pipeline Demonstration
Pipeline feature validated.

2. Signal Demonstration
Signal demonstration: SIGINT received.

3. Process Demonstration
Child process running. PID = 982
Child process completed.

Final project review completed.
All demonstration scenarios executed.
Parent received child completion.

Final project review completed.
All demonstration scenarios executed.
adarohan18@Rohith:~$ |
```

